Naphat Amundsen presents:

Muon

Moment**U**m **O**rthogonalized by **N**ewton-Schulz

2026-03-25

Intro

- AdamW and similar optimizers essentially do momentum on each weight individually.
  - Empirically shown that the momentum matrix to have a tendency to become low-rank.
- Muon takes a more wholistic view of the momentum values together

Standard AdamW

Require : γ (lr), β_1, β_2 (betas), θ_0 (params), $f(\theta)$ (objective), ε (epsilon), λ (weight decay)

Initialize: $m_0 \leftarrow 0$ (first moment), $v_0 \leftarrow 0$ (second moment), $v_0^{\max} \leftarrow 0$
for $t = 1$ to ... **do**
  Compute gradient $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
  Weight decay $\theta_t \leftarrow \theta_{t-1} - \gamma\lambda\theta_{t-1}$
  Update momentum $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$
  Update RMS $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
  Bias correct momentum $\hat{m}_t \leftarrow m_t / (1-\beta_1^t)$
  Bias correct RMS $\hat{v}_t \leftarrow v_t / (1-\beta_2^t)$
  Update parameters $\theta_t \leftarrow \theta_t - \gamma \hat{m}_t / \left(\sqrt{\hat{v}_t} + \varepsilon\right)$
return θ_t

Understanding the issue

AdamW (and other momentum-based) optimizers only look at each weights individually.

- Let's look closer at momentum update in the case of a (N, M) matrix:

$$m \leftarrow \beta m + \frac{\partial L}{\partial \theta}, \quad m = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \\ m_{41} & m_{42} & m_{43} \end{bmatrix}$$

what is observed is that the momentum matrix during training have tendencies to become "practically low-rank": e.g.

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0.99 & 1.001 \\ 1 & 1.01 & 1.002 \\ 1 & 1.01 & 1.003 \end{bmatrix}$$

If we think of gradient descent as a ball rolling down the loss landscape, the problem is that the ball tends build momentum and “shoot” in a certain direction.

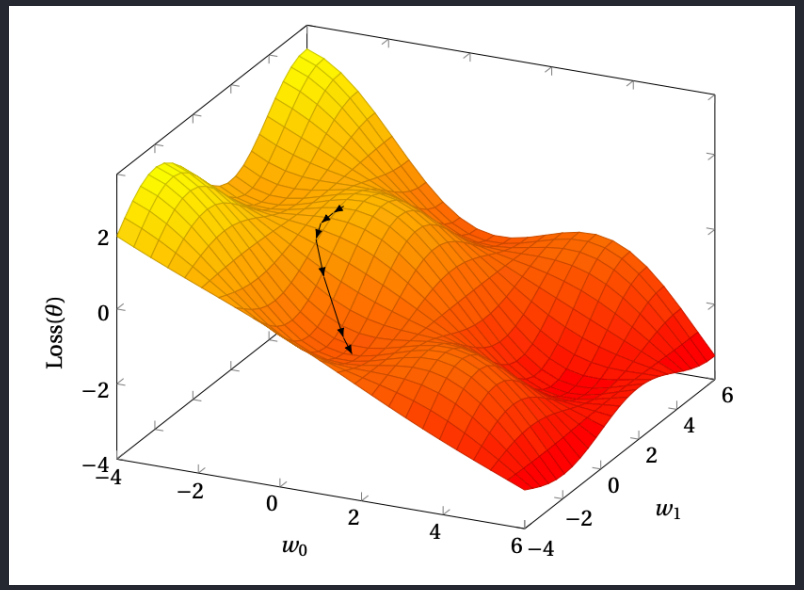

Figure 1: Visualization of gradient steps over a loss landscape

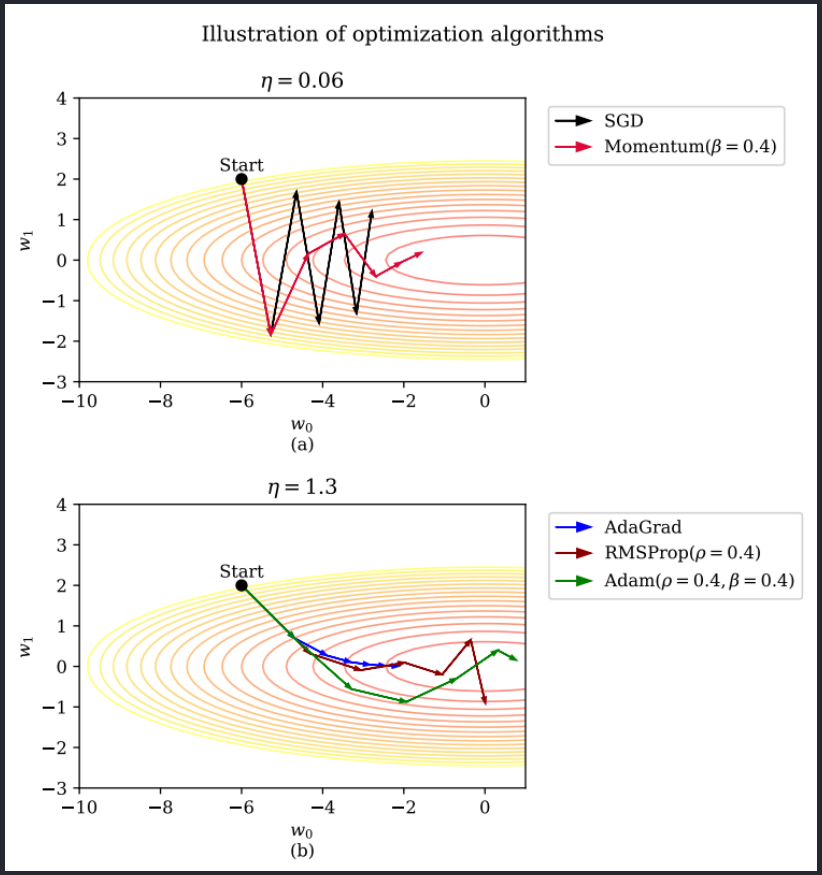

Figure 2: Visualization of gradient steps in a "bowl"

So, how can one condition the momentum matrix to be more well-behaved?

- The regular "element-by-element" optimizers cannot solve this issue directly, because they have to look at the momentums more wholistically, which they don't.
- Thus have a method that "looks at the whole matrix".
- Muon: let's orthogonalize the momentum matrix!

Orthogonalization recap

$\text{Ortho}(\boldsymbol{M}) = \text{argmin}_{\boldsymbol{O}}\{\|\boldsymbol{O} - \boldsymbol{M}\|_F\}$ such that $OO^T = I$ or $O^TO = I$

Rant: the naming is stupid, the name is called "orthogonal", but the mathematical definition here implies orthonormal.

- In English: find an orthonormal matrix that is as similar to $\boldsymbol{M}$ as possible
- A theoretically sound solution is to use singular value decomposition (SVD):
  - Factorize the matrix M to the form $M = U\Sigma V^T$
  - Where U is a unitary matrix: $U^TU = UU^T = I$
  - One can then do UV^T to get an orthonormal version of M

Problem solved, right?

WAKE UP, YOU'RE NOT IN A MOVIE
THIS IS REALITY SLAPPING YOU IN THE FACE
imgflip.com

Problem: Calculating SVD for every single backprop for every single matrix in a model is too resource intensive.

They do a trick: use Newton-Schulz iteration to approximate the orthogonalization, which is much more efficient and can be easily accelerated by GPUs.

Newton-Schulz 101

The Newton-Schulz iteration is a mathematical approximation to approximate the Matrix Sign Function.

For Scalars (x): The sign function $\text{sgn}(x)$ strips away the magnitude and keeps only the direction (returning -1, 0, or $+1$).

For Matrices (M): The sign function $\text{sgn}(M)$ strips away the stretching factors (singular values) and keeps only the orientation.

The SVD Connection:
- Given the decomposition $M = U\Sigma V^T$, the "sign" of M is defined as UV^T.
- If we force Σ to become the Identity Matrix I, we essentially get the "sign" of M.

Newton-Schulz iteration: Newton-Schulz iteration is a method to approximate the matrix sign function without explicitly computing the SVD.
- It uses an **odd polynomial** (e.g., $1.5X - 0.5X^3$) to iteratively "squash" the singular values in Σ toward 1.
- Because the polynomial is **odd**, it preserves the direction of the gradient while only using matrix multiplications.

The Geometric Result: UV^T is the closest orthonormal matrix to the original gradient. It turns a "squashed ellipse" (an ill-conditioned gradient) into a "perfect sphere" (an orthonormal update).

The Muon Optimizer

Require: Learning rate η, momentum μ

Initialize $B_0 \leftarrow 0$
for $t = 1, \ldots$ **do**
  Compute gradient $G_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_{t-1})$
  $B_t \leftarrow \mu B_{t-1} + G_t$
  $O_t \leftarrow \text{NewtonSchulz5}(B_t)$
  Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
end for
return θ_t

Note: there are some differences between the pseudocode and the actual implementation

Practical limitations of Muon:

- Only works for 2D matrices, so it is only applied to the weight matrices in the model, not the bias or other parameters.

From the author:

> - Will Muon scale to larger trainings? (e.g., 20B+ parameters for 1T+ tokens)
> - Will it be possible to properly distribute the Newton-Schulz iterations used by Muon across a large-scale GPU cluster?
> - Is it possible that Muon works only for pretraining, and won't work for finetuning or reinforcement learning workloads?
>
> At the time of writing, I don't know the answers to these questions.
>
> — Keller Jordan

See https://kellerjordan.github.io/posts/muon/ for a more detailed writeup

Illustration. See code.